

Place agri_yield_dataset.csv in your notebook working directory and run:

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler

# Load the dataset
df = pd.read_csv("agri_yield_dataset.csv")
# 1. Preview top 5 rows
print("--- First 5 Records ---")
display(df.head())
# 2. Check structure and column data types
print("--- Dataset Info ---")
df.info()
# 3. Statistical summary of numerical columns
print("--- Summary Statistics ---")
display(df.describe())
```

--- First 5 Records ---

	State	Crop	Soil_Type	Fertilizer	N	P	K	Rainfall_mm	Temperature_C	Yield_kg_per_acre
0	Karnataka	Soybean	Loamy	DAP	96	41	51	123	31.06	
1	Odisha	Cotton	Red Soil	Urea	29	36	112	247	33.97	
2	Punjab	Groundnut	Red Soil	Compost	38	37	177	142	24.21	
3	Gujarat	Wheat	Red Soil	Compost	58	77	129	227	30.85	
4	Andhra Pradesh	Cotton	Clay	Organic	108	61	63	263	37.81	

--- Dataset Info ---

<class 'pandas.DataFrame'>

RangeIndex: 1500 entries, 0 to 1499

Data columns (total 12 columns):

#	Column	Non-Null Count	Dtype
0	State	1500 non-null	str
1	Crop	1500 non-null	str
2	Soil_Type	1500 non-null	str
3	Fertilizer	1500 non-null	str
4	N	1500 non-null	int64
5	P	1500 non-null	int64
6	K	1500 non-null	int64
7	Rainfall_mm	1500 non-null	int64
8	Temperature_C	1500 non-null	float64
9	Yield_kg_per_acre	1500 non-null	int64
10	Soil_pH	1500 non-null	float64
11	Year	1500 non-null	int64

dtypes: float64(2), int64(6), str(4)

memory usage: 140.8 KB

--- Summary Statistics ---

	N	P	K	Rainfall_mm	Temperature_C	Yield_kg_per_acre
count	1500.000000	1500.000000	1500.000000	1500.000000	1500.000000	1500.000000
mean	74.968000	61.826000	100.374667	174.620667	28.054027	5771.086000
std	34.631063	30.092049	51.109814	67.124555	5.493560	12063.581091
min	10.000000	5.000000	10.000000	50.000000	18.020000	505.000000
25%	50.000000	40.000000	62.000000	127.750000	23.932500	1272.750000
50%	75.000000	61.000000	101.000000	175.000000	27.955000	2065.500000
75%	99.000000	84.000000	135.250000	225.000000	32.240000	2905.250000
max	139.000000	119.000000	199.000000	299.000000	37.960000	88401.000000

Confirm there are zero missing records and no duplicated entries

```
In [3]: # Check for missing values
print("Missing values per column:", df.isnull().sum())
# Check for duplicate rows
duplicates_count = df.duplicated().sum()
print(f"Duplicate records count: {duplicates_count}")
```

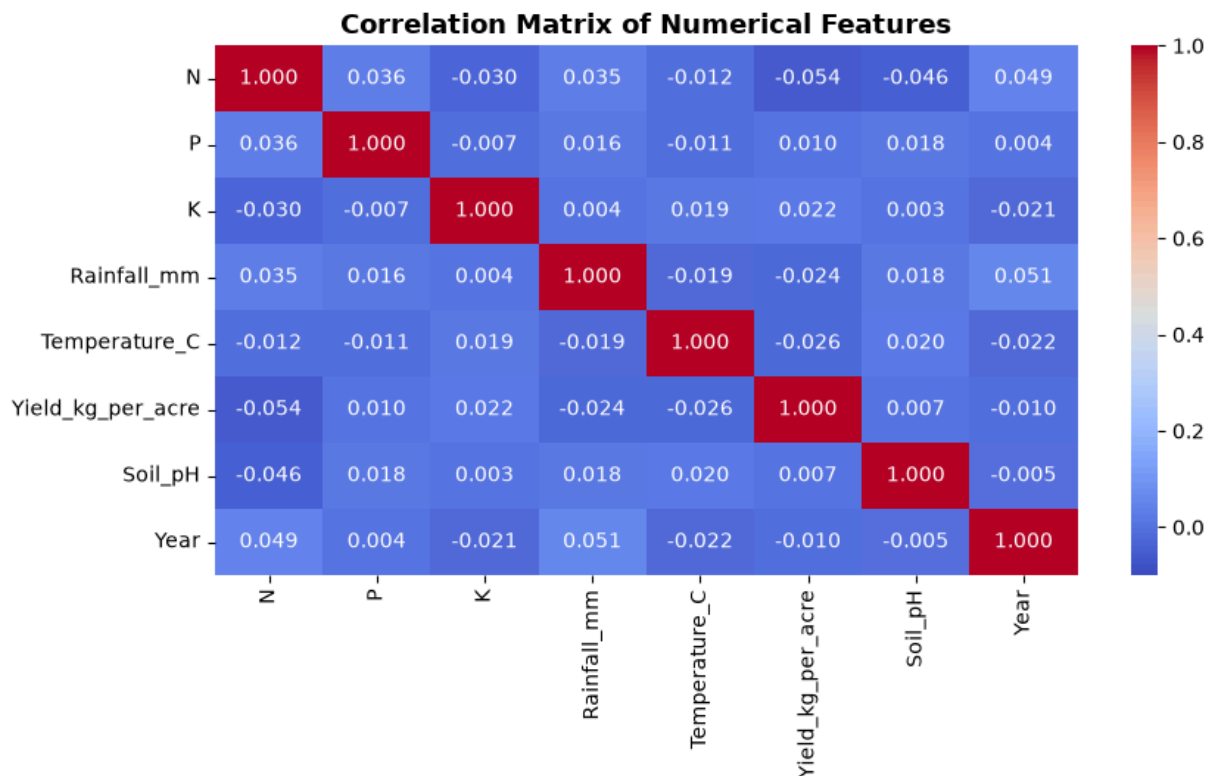
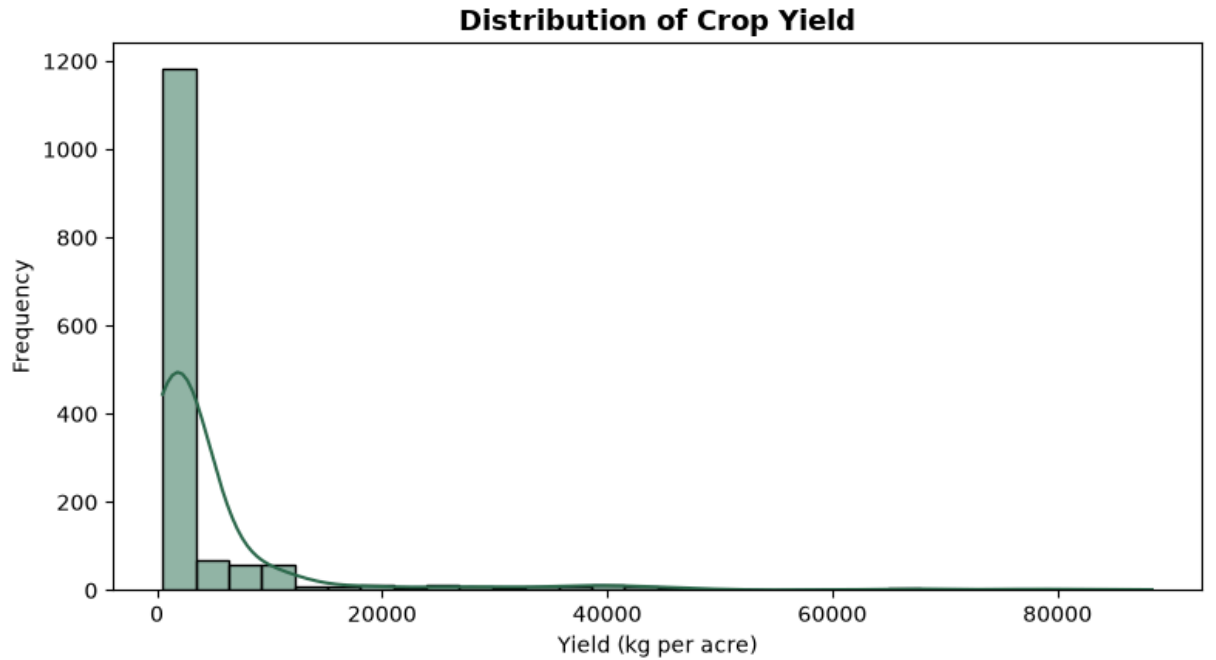
```
Missing values per column: State          0
Crop          0
Soil_Type     0
Fertilizer    0
N             0
P             0
K             0
Rainfall_mm   0
Temperature_C 0
Yield_kg_per_acre 0
Soil_pH       0
Year         0
dtype: int64
Duplicate records count: 0
```

Generate the distribution plot for the target variable and the correlation heatmap for numerical attributes

```
In [4]: # Graph 1: Distribution of Crop Yield
plt.figure(figsize=(8, 4.5))
sns.histplot(df['Yield_kg_per_acre'], bins=30, kde=True, color='#2d6a4f')
plt.title("Distribution of Crop Yield", fontsize=13, weight='bold')
plt.xlabel("Yield (kg per acre)")
plt.ylabel("Frequency")
plt.tight_layout()
plt.show()

# Graph 2: Numerical Correlation Heatmap
plt.figure(figsize=(9, 5.5))
```

```
sns.heatmap(df.corr(numeric_only=True), annot=True, fmt=".3f", cmap='coolwarm', vmi
plt.title("Correlation Matrix of Numerical Features", fontsize=13, weight='bold')
plt.tight_layout()
plt.show()
```



Separate target y from features X, apply one-hot encoding to convert text categories, and scale numerical variables using StandardScaler

```
In [5]: # Separate features and target
X = df.drop("Yield_kg_per_acre", axis=1)
y = df["Yield_kg_per_acre"]
```

```

# One-Hot Encoding on categorical features (drops first level to avoid collinearity)
X_encoded = pd.get_dummies(X, drop_first=True)
print("Shape after One-Hot Encoding:", X_encoded.shape)
# Standardize numerical features
scaler = StandardScaler()
numerical_cols = ['N', 'P', 'K', 'Rainfall_mm', 'Temperature_C', 'Soil_pH', 'Year']
X_encoded[numerical_cols] = scaler.fit_transform(X_encoded[numerical_cols])
print("Final dataset shape ready for ML training:", X_encoded.shape)
display(X_encoded.head())

```

Shape after One-Hot Encoding: (1500, 30)

Final dataset shape ready for ML training: (1500, 30)

	N	P	K	Rainfall_mm	Temperature_C	Soil_pH	Year	State_C
0	0.607519	-0.692307	-0.966373	-0.769285	0.547364	0.057348	-1.224843	
1	-1.327806	-0.858520	0.227534	1.078644	1.077252	-0.553163	-1.360988	
2	-1.067837	-0.825277	1.499729	-0.486134	-0.699967	0.414721	0.408886	
3	-0.490128	0.504421	0.560262	0.780591	0.509125	-1.267907	1.361895	
4	0.954144	-0.027458	-0.731506	1.317086	1.776485	-0.806301	0.681175	

5 rows × 30 columns



In []: